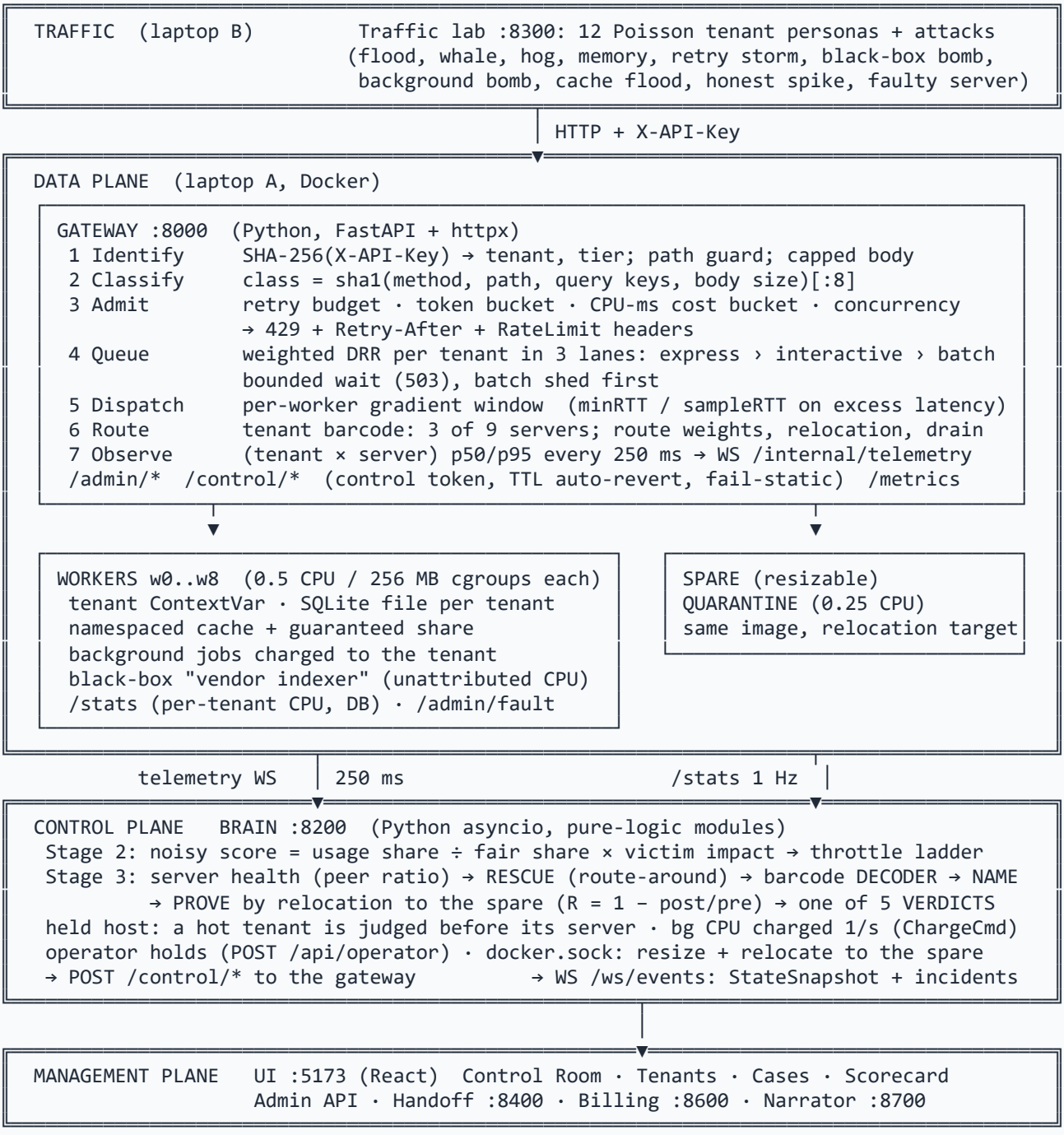
TenantGuard

Multi-Tenant Isolation & Noisy-Neighbor Defense · CodeAstra 2.0 · Cloud Computing & Distributed Systems · PS 3

Architecture | Tech Stack | Project Flow

> *"TenantGuard gives every tenant of a shared service its fair share, and names the tenant behind any slowdown, even when the expensive work is invisible to a rate limiter."*

Version 1: Technical Architecture



The 3 Planes

Plane	Components	Job
Data	gateway, w0..w8, spare, quarantine	Serve every tenant request: identify, admit, queue fairly, route by barcode, measure
Control	brain	Watch telemetry + worker stats; rescue victims, name the culprit, prove it by relocation, give a verdict, throttle with TTLs, charge background CPU, explain every action
Management	admin API, UI, handoff bridge, billing, narrator, lab, demo driver	Operators see and steer it: modes, quotas, cases, attacks, scorecard, fair bills, incident reports

The 5 Layers of Isolation (not just a rate limit)

Layer	Name	What it does	Where
L1	Request	Identity, token bucket, retry budget: a flood or retry storm gets 429 with an accurate Retry-After	`gateway/admission.py`, `retry.py`
L2	Cost	A CPU-ms bucket debited with the CPU each request really used (measured by the worker)	`gateway/buckets.py`
L3	Scheduler	Weighted DRR in 3 lanes behind an adaptive per-worker window: cheap requests never wait behind a whale	`gateway/scheduler.py`, `limiter.py`
L4	Kernel	cgroup CPU / memory / pids limits per worker container; spare and quarantine pools	`docker-compose.yml`
L5	Data	One SQLite file per tenant; namespaced cache with a guaranteed share; per-tenant memory budget	`worker/app/store.py`, `cache.py`
+ Adaptive loop		The brain: rescue, name, prove, judge, throttle, recover. It works even when the harm is invisible to L1 and L2.	

Components & Tech Stack

Component	Port	Built with	Notes
Gateway	8000	Python 3.12, FastAPI, httpx (pooled), uvicorn	All state in memory (one gateway); Prometheus `/metrics`
Workers ×9 + 2	8100	Python, FastAPI (sync handlers), SQLite, psutil	`time.thread_time()` per request = exact handler CPU
Brain	8200	Python asyncio, websockets, httpx, Pydantic	Pure-logic modules (score, ladder, health, rescue, decoder, verdicts) tested without I/O; the prover talks to the Docker socket
Traffic lab	8300	Python asyncio + httpx keep-alive pools	Writes `lab/results/*.json`: the only source of scorecard numbers
UI	5173	React 18, Vite, TypeScript, Tailwind, Recharts	WS from the brain at 2 Hz; mock mode for offline development
Handoff bridge	8400	Python, FastAPI, websockets	Web PS3 convergence: incidents ↔ cases
Billing	8600	Python, FastAPI, httpx	Fair bills at calm reference prices, SLA credits, "Why was I slow?"
Narrator	8700	Python, FastAPI, templates	Deterministic incident report + anonymised customer message
Contracts + config		Pydantic models in `common/`, `tenantguard.yaml`	One shared schema; every tunable in YAML
Infra		Docker Compose, cgroup v2, two laptops over Tailscale	No external network at runtime

Adaptive Throttling Ladder (Stage 2)

Level	Enters when	Action (always with a TTL)
normal	-	Full plan
warn	noisy score ≥ 1.5, use ≥ 1.6× fair share and ≥ 4 neighbours hurt, held 5 s	Incident opened, nothing throttled
throttle	score ≥ 2.0, held 2 s	Quota × 0.5 per step (AIMD), via LimitsCmd
deprioritize	still noisy, held 4 s	Scheduler weight × 0.25
quarantine	score ≥ 4.0, held 6 s	Quota × 0.1 and weight × 0.1; a block needs human approval (handoff case)
recover	score < 1.0, held 10 s	+0.1 factor per step back to the full plan

Thresholds and hold times come from `tenantguard.yaml` (`brain.noisy`). Rescue comes first: the ladder acts only if harm persists. A legit burst that hurts nobody is never throttled.

Stage 3: Rescue, Name, Prove

Step	How
Health	Each server's p95 compared with its peers (median over its tenants, so a whale's own slow requests don't count). Calibrated on the Docker host: the signal must be sustained over a 5 s window, and a server is never "slow" below the larger of 25 ms and half the strictest SLA on it
Rescue	Route weights move victims off slow servers onto their healthy ones, keeping every server under 70% CPU
Name	AG(2,3) barcodes: any two tenants share at most one server, so the set of slow servers decodes to the culprit (with a confidence) or to a host fault with no tenant blamed. A slow server where a hot tenant runs is *held*: that tenant is judged before the server is suspected
Prove	Move the suspect to the spare: the original servers recover and the slowdown follows it (R = 1 − post/pre slowdown). `DockerProver` sizes the spare through the Docker socket; a suspect host is drained and measured idle
Verdict	`mitigated_no_blame` (not proven: nobody penalised) · `over_plan` (the ladder tightens limits) · `costly_pattern` (only the class that grew goes to quarantine) · `honest_growth` (keeps the spare as a burst lane, no penalty) · `host_fault` (host stays drained). TTL-bound, limited by an action budget

Safety

Rule	How
Auto-undo	Every control override has a finite TTL (≤ `gateway.ttl_max_s`) and reverts by itself; the brain renews it while needed
Fail-static	If brain heartbeats stop for 2 s, the gateway keeps its last config while TTLs still expire. A dead brain never leaves a tenant punished
Humans and ladder agree	Operator release / extend go to the brain (`OperatorCmd`), so the ladder obeys them instead of fighting them
Hardened edges	Control token on `/admin/*`, `/control/*`; path guard (no traversal to worker admin); bounded TTLs and quotas; capped request bodies

Measured: why a rate limiter is not enough

Endpoint	Handler CPU (what request metering sees)	Background CPU	Black-box indexer CPU
`GET /api/light`	0.396 ms	0	0
`POST /api/write`	0.185 ms	1.429 ms	0
`POST /api/ingest`	**0.066 ms**	0	**20.25 ms**

Source: `worker/calibration.json`. Docker host, one worker limited to 0.5 CPU, sequential client, mean per request.

Measured: the 3-mode scorecard

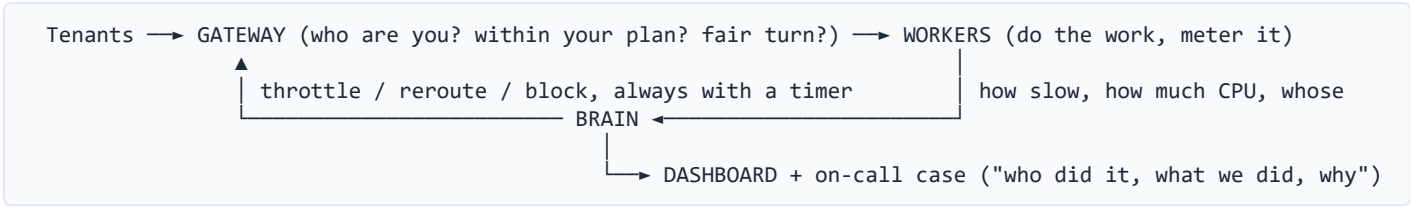
Scenario	OFF	Industry	TenantGuard
whale	149 → **691** → 254	157 → 176 → 165	158 → 183 → 154
flood	200 → 162 → 186	166 → 164 → 163	143 → 144 → 175

Median victim p99 in ms, baseline → attack → after; 3 runs per cell on the Docker host (`lab/results/summary-scorecard-host.json` + 18 run files). No innocent tenant was penalised in any run. Both attacks are contained by the gateway limits that Industry and TenantGuard share, so the two are about equal here. One OFF flood run recorded 1,522 refused requests, most likely carried over from a mode switch.

Host acceptance run (2026-10-13)	What the brain did
Idle, then calm traffic	0 events; 0 ladder moves (2 real slowdowns on w7 routed around, no tenant blamed)
Big-plan whale K	Named at 99%, proved by relocation with R = 0.88, verdict `honest_growth`; held hosts w1/w8 judged first; victims rerouted
Faulty server w4	`host_fault` at 5.1× idle latency, no tenant blamed
End	0 overrides left

Version 2: Simple Project Flow (Team View)

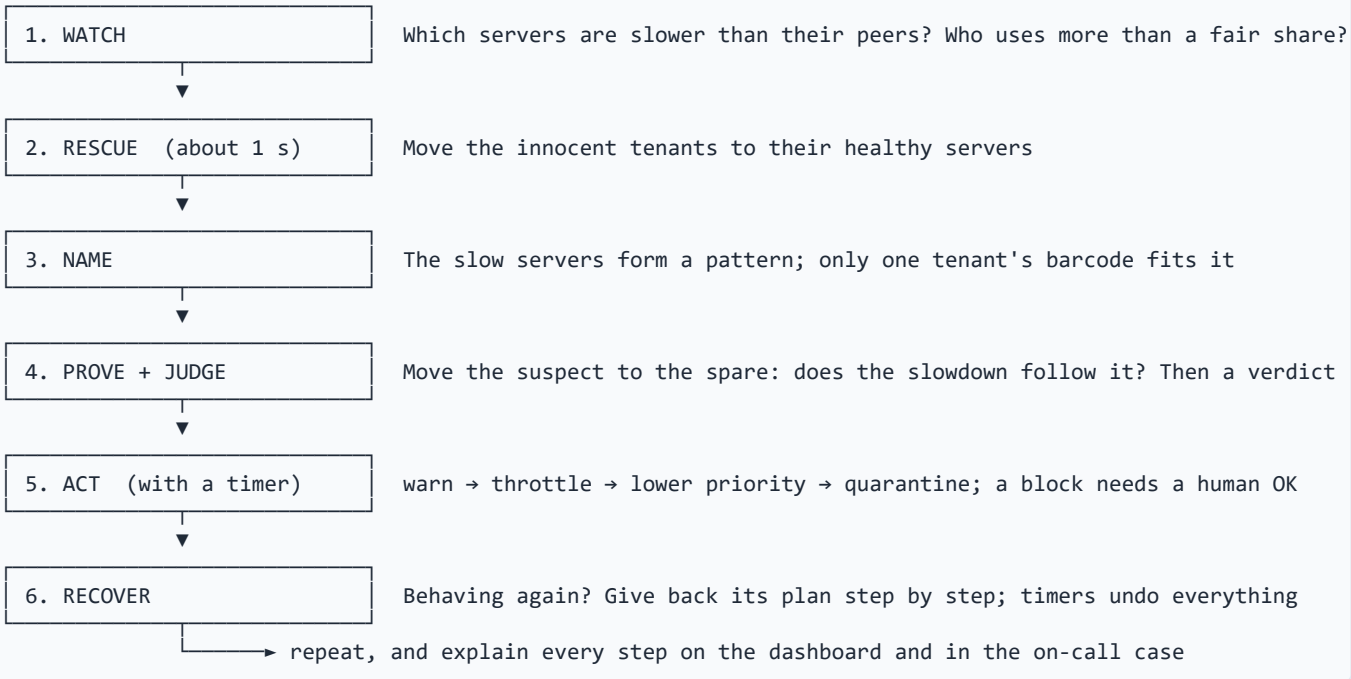
The Big Picture in One Line



A Request's Journey, Step by Step

1. Tenant K sends POST /api/ingest with its API key
2. Gateway: "You are K (Pro plan)." Unknown key? → 401
3. Within your plan? rate ✓ CPU budget ✓ in-flight ✓ not a retry storm ✓ (else 429 + "retry in 2 s")
4. Wait your fair turn: cheap requests take the express lane; nobody waits more than 1 s (else 503)
5. Go to one of K's 3 servers (its "barcode"), avoiding slow or drained ones
6. Worker does the work in K's own database and charges the CPU to K
   ...but the black-box indexer it triggers burns 300× more CPU that nobody bills
7. Gateway records the latency for (K, server) and reports it to the brain every 250 ms

The Automatic Protection Loop (4 times a second)



Which Box Does What

Box	In one sentence
Gateway	The front door: knows every tenant, enforces its plan, keeps the queue fair, and obeys the brain's timed orders.
Workers	The shared app: does the work, keeps each tenant's data apart, and measures exactly what each tenant costs.
Brain	The detective: spots slow servers, rescues victims, names the culprit, proves it, decides the verdict, and turns the throttle up or down.
Dashboard	The control room: every tenant, every server, every incident and the reason for every action, live.
Lab	The crowd: 12 realistic tenants plus ten attacks, and the measured scorecard.
Handoff bridge	Turns each incident into a case for the on-call engineer and carries their decision back.
Billing	Bills each tenant fairly: victims never pay for contention, SLA misses earn credits, and each tenant can ask "Why was I slow?".
Narrator	Writes the incident report and a customer message that names nobody.

The Demo in Pictures (4 minutes, `tools/demo.py`)

Act	What the judges see
Calm	12 green tiles, fairness near 1
OFF + flood	Everyone sharing K's servers turns red: the noisy-neighbour problem
Industry	The flood is refused (429). Then the black-box bomb: cheap-looking ingest calls that trigger heavy hidden indexer CPU; the rate limiter barely sees it
TenantGuard	Same bomb: victims rescued, K named with a confidence, proved by relocation, a verdict; the case appears for the on-call engineer
Faulty server	One server goes bad by itself (the lab's `faulty_server` attack injects the fault): host fault, no tenant blamed
Honest spike	An Enterprise tenant's sale day: a genuine burst that hurts nobody, so it is not throttled
Scorecard	OFF vs Industry vs TenantGuard victim p99, measured on the Docker host for whale and flood (from `lab/results`)

Presentation tip: Put Version 1 on the architecture slide, then walk the judges through it using the Version 2 story ("let's follow one request from K..."). Judges remember stories better than boxes.